

第2部分 基础篇

第6章 数据结构基础

学习目标

- ☑ 了解双端队列，能用栈进行简单的表达式解析
- ☑ 熟练掌握链表的数组实现及测试方法
- ☑ 掌握对比测试的方法
- ☑ 掌握完全二叉树的数组实现
- ☑ 掌握二叉树的链式表示法和数组表示法
- ☑ 了解动态内存分配和释放方法及其注意事项
- ☑ 理解内存池的作用以及一种简易实现方法
- ☑ 掌握二叉树的先序、后序、中序遍历和层次遍历
- ☑ 掌握图的 DFS 及连通块计数
- ☑ 掌握图的 BFS 及最短路的输出
- ☑ 掌握拓扑排序算法
- ☑ 掌握欧拉回路算法

本章介绍基础数据结构，包括线性表（包括栈、队列、链表）、二叉树和图。尽管这些内容本身并不算“高级”，但却是很多高级内容的基础。如果数据结构基础没有打好，很难设计出正确、高效的算法。

6.1 再谈栈和队列

例题 6-1 并行程序模拟 (Concurrency Simulator, ACM/ICPC World Finals 1991, UVa210)

你的任务是模拟 n 个程序（按输入顺序编号为 $1 \sim n$ ）的并行执行。每个程序包含不超过 25 条语句，格式一共有 5 种：var = constant（赋值）；print var（打印）；lock；unlock；end。

变量用单个小写字母表示，初始为 0，为所有程序公有（因此在一个程序里对某个变量赋值可能会影响另一个程序）。常数是小于 100 的非负整数。

每个时刻只能有一个程序处于运行态，其他程序均处于等待态。上述 5 种语句分别需要 t_1 、 t_2 、 t_3 、 t_4 、 t_5 单位时间。运行态的程序每次最多运行 Q 个单位时间（称为配额）。当一个程序的配额用完之后，把当前语句（如果存在）执行完之后该程序会被插入一个等待队列中，然后处理器从队首取出一个程序继续执行。初始等待队列包含按输入顺序排列

的各个程序，但由于 lock/unlock 语句的出现，这个顺序可能会改变。

lock 的作用是申请对所有变量的独占访问。lock 和 unlock 总是成对出现，并且不会嵌套。lock 总是在 unlock 的前面。当一个程序成功执行完 lock 指令之后，其他程序一旦试图执行 lock 指令，就会马上被放到一个所谓的阻止队列的尾部（没有用完的配额就浪费了）。当 unlock 执行完毕后，阻止队列的第一个程序进入等待队列的首部。

输入 $n, t1, t2, t3, t4, t5, Q$ 以及 n 个程序，按照时间顺序输出所有 print 语句的程序编号和结果。

【分析】

因为有“等待队列”和“阻止队列”的字眼，本题看上去是队列的一个简单应用，但请注意这句话：“阻止队列的第一个程序进入等待队列的首部”。这违反了队列的规则：新元素插入了队列首部而非尾部。

有两个方法可以解决这个问题：一是放弃 STL 队列，自己写一个支持“首部插入”的“队列”：用两个变量 front 和 rear 代表队列当前首尾下标，则传统的入队和出队分别是 $q[++rear] = x$ 和 $x = q[front++]$ ，而“插入到队首”则是 $q[--front] = x$ 。细心的读者应该已经发现：如果 $front=0$ ，则“插入到队首”会产生越界错误。确实如此，不过好在本题不会出现这样的情况（想一想，为什么）。

第二种方法是使用 STL 中的“双端队列”deque。它可以支持快速地在首尾两端进行插入和删除，有兴趣的读者可以自行查阅 STL 文档。

提示 6-1：如果要在“队列”两端进行插入和删除，可以用 STL 中的双端队列 deque。

例题 6-2 铁轨 (Rails, ACM/ICPC CERC 1997, UVa 514)

某城市有一个火车站，铁轨铺设如图 6-1 所示。有 n 节车厢从 A 方向驶入车站，按进站顺序编号为 $1 \sim n$ 。你的任务是判断是否能让它们按照某种特定的顺序进入 B 方向的铁轨并驶出车站。例如，出栈顺序 (5 4 1 2 3) 是不可能的，但 (5 4 3 2 1) 是可能的。

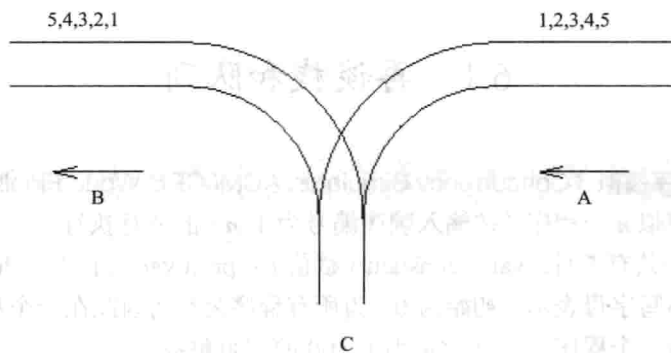


图 6-1 铁轨

为了重组车厢，你可以借助中转站 C。这是一个可以停放任意多节车厢的车站，但由于末端封顶，驶入 C 的车厢必须按照相反的顺序驶出 C。对于每个车厢，一旦从 A 移入 C，就不能再回到 A 了；一旦从 C 移入 B，就不能回到 C 了。换句话说，在任意时刻，只有两种选择： $A \rightarrow C$ 和 $C \rightarrow B$ 。

【分析】

在中转站 C 中, 车厢符合后进先出的原则, 因此是一个栈。代码如下:

```
#include<cstdio>
#include<stack>
using namespace std;
const int MAXN = 1000 + 10;

int n, target[MAXN];

int main(){
    while(scanf("%d", &n) == 1){
        stack<int> s;
        int A = 1, B = 1;
        for(int i = 1; i <= n; i++){
            scanf("%d", &target[i]);
            int ok = 1;
            while(B <= n){
                if(A == target[B]){ A++; B++; }
                else if(!s.empty() && s.top() == target[B]){ s.pop(); B++; }
                else if(A <= n) s.push(A++);
                else { ok = 0; break; }
            }
            printf("%s\n", ok ? "Yes" : "No");
        }
        return 0;
    }
}
```

栈对于表达式求值有着特殊的作用。下面举一例。

例题 6-3 矩阵链乘 (Matrix Chain Multiplication, UVa 442)

输入 n 个矩阵的维度和一些矩阵链乘表达式, 输出乘法的次数。如果乘法无法进行, 输出 error。假定 A 是 $m*n$ 矩阵, B 是 $n*p$ 矩阵, 那么 AB 是 $m*p$ 矩阵, 乘法次数为 $m*n*p$ 。如果 A 的列数不等于 B 的行数, 则乘法无法进行。

例如, A 是 $50*10$ 的, B 是 $10*20$ 的, C 是 $20*5$ 的, 则 $(A(BC))$ 的乘法次数为 $10*20*5$ (BC 的乘法次数) + $50*10*5$ ((A(BC))的乘法次数) = 3500。

【分析】

本题的关键是解析表达式。本题的表达式比较简单, 可以用一个栈来完成: 遇到字母时入栈, 遇到右括号时出栈并计算, 然后结果入栈。因为输入保证合法, 括号无须入栈。

提示 6-2: 简单的表达式解析可以借助栈来完成。

这里直接给出代码, 其中的道理请读者细细体会。

```
#include<cstdio>
#include<stack>
#include<iostream>
#include<string>
using namespace std;

struct Matrix {
    int a, b;
    Matrix(int a=0, int b=0):a(a),b(b) {}
} m[26];

stack<Matrix> s;

int main() {
    int n;
    cin >> n;
    for(int i = 0; i < n; i++) {
        string name;
        cin >> name;
        int k = name[0] - 'A';
        cin >> m[k].a >> m[k].b;
    }
    string expr;
    while(cin >> expr) {
        int len = expr.length();
        bool error = false;
        int ans = 0;
        for(int i = 0; i < len; i++) {
            if(isalpha(expr[i])) s.push(m[expr[i] - 'A']);
            else if(expr[i] == '(') {
                Matrix m2 = s.top(); s.pop();
                Matrix m1 = s.top(); s.pop();
                if(m1.b != m2.a) { error = true; break; }
                ans += m1.a * m1.b * m2.b;
                s.push(Matrix(m1.a, m2.b));
            }
        }
        if(error) printf("error\n"); else printf("%d\n", ans);
    }
    return 0;
}
```

6.2 链 表

到目前为止,已经大量地使用过了数组及其不定长版本——vector,使用的方法大都是随机存取和往末尾添加/删除元素。但有时也需要向数组中插入元素,下面便是一例。

例题 6-4 破损的键盘(又名:悲剧文本)(Broken Keyboard(a.k.a. Beiju Text), UVa 11988)

你有一个破损的键盘。键盘上的所有键都可以正常工作,但有时 Home 键或者 End 键会自动按下。你并不知道键盘存在这一问题,而是专心地打稿子,甚至连显示器都没打开。当你打开显示器之后,展现在你面前的是一段悲剧的文本。你的任务是在打开显示器之前计算出这段悲剧文本。

输入包含多组数据。每组数据占一行,包含不超过 100000 个字母、下划线、字符 “[” 或者 “]”。其中字符 “[” 表示 Home 键,“]” 表示 End 键。输入结束标志为文件结束符 (EOF)。输入文件不超过 5MB。对于每组数据,输出一行,即屏幕上的悲剧文本。

样例输入:

```
This_is_a_[Beiju]_text
[[]][[]]Happy_Birthday_to_Tsinghua_University
```

样例输出:

```
BeijuThis_is_a__text
Happy_Birthday_to_Tsinghua_University
```

【分析】

最简单的想法便是用数组来保存这段文本,然后用一个变量 pos 保存“光标位置”。这样,输入一个字符相当于在数组中插入一个字符(需要先把后面的字符全部右移,给新字符腾出位置)。

很可惜,这样的代码会超时。为什么?因为每输入一个字符都可能会引起大量字符移动。在极端情况下,例如,2500000 个 a 和 “[” 交替出现,则一共需要 $0+1+2+\cdots+2499999=6*10^{12}$ 次字符移动。

解决方案是采用链表(linked list)。每输入一个字符就把它存起来,设输入字符串是 $s[1\sim n]$,则可以用 $next[i]$ 表示在当前显示屏中 $s[i]$ 右边的字符编号(即在 s 中的下标)^①。

提示 6-3: 在数组中频繁移动元素是很低效的,如有可能,可以使用链表。

为了方便起见,假设字符串 s 的最前面还有一个虚拟的 $s[0]$,则 $next[0]$ 就可以表示显示屏中最左边的字符。再用一个变量 cur 表示光标位置:即当前光标位于 $s[cur]$ 的右边。cur=0 说明光标位于“虚拟字符” $s[0]$ 的右边,即显示屏的最左边。

提示 6-4: 为了方便起见,常常在链表的第一个元素之前放一个虚拟结点。

^① 读者可能其他数据结构书中见过基于指针的链表实现方式,但是链表并不一定要用指针。

为了移动光标，还需要用一个变量 `last` 表示显示屏的最后一个字符是 `s[last]`。代码如下：

```
#include<cstdio>
#include<cstring>
const int maxn = 100000 + 5;
int last, cur, next[maxn]; //光标位于 cur 号字符的后面
char s[maxn];

int main() {
    while(scanf("%s", s+1) == 1) {
        int n = strlen(s+1); //输入保存在 s[1], s[2]...中
        last = cur = 0;
        next[0] = 0;
        for(int i = 1; i <= n; i++){
            char ch = s[i];
            if(ch == '[') cur = 0;
            else if(ch == ']') cur = last;
            else {
                next[i] = next[cur];
                next[cur] = i;
                if(cur == last) last = i; //更新“最后一个字符”编号
                cur = i; //移动光标
            }
        }
        for(int i = next[0]; i != 0; i = next[i])
            printf("%c", s[i]);
        printf("\n");
    }
    return 0;
}
```

例题 6-5 移动盒子 (Boxes in a Line, UVa 12657)

你有一行盒子，从左到右依次编号为 $1, 2, 3, \dots, n$ 。可以执行以下 4 种指令：

- 1 XY 表示把盒子 X 移动到盒子 Y 左边（如果 X 已经在 Y 的左边则忽略此指令）。
- 2 XY 表示把盒子 X 移动到盒子 Y 右边（如果 X 已经在 Y 的右边则忽略此指令）。
- 3 XY 表示交换盒子 X 和 Y 的位置。
- 4 表示反转整条链。

指令保证合法，即 $X \neq Y$ 。例如，当 $n=6$ 时在初始状态下执行 114 后，盒子序列为 231456。接下来执行 235，盒子序列变成 214536。再执行 316，得到 264531。最终执行 4，得到 135462。

输入包含不超过 10 组数据，每组数据第一行为盒子个数 n 和指令条数 m ($1 \leq n, m \leq$

100000)，以下 m 行每行包含一条指令。每组数据输出一行，即所有奇数位置的盒子编号之和。位置从左到右编号为 $1 \sim n$ 。

样例输入：

```
6 4
1 1 4
2 3 5
3 1 6
4
6 3
1 1 4
2 3 5
3 1 6
100000 1
4
```

样例输出：

```
Case 1: 12
Case 2: 9
Case 3: 2500050000
```

【分析】

根据前面的经验，如果用数组来保存盒子，肯定会超时，但如果像例题 6-4 那样只保存一个 next 值，似乎又不够，怎么办？

解决方法是采用双向链表（doubly linked list）：用 $\text{left}[i]$ 和 $\text{right}[i]$ 分别表示编号为 i 的盒子左边和右边的盒子编号（如果是 0，表示不存在），则下面的过程可以让两个结点相互连接：

```
void link(int L, int R) {
    right[L] = R; left[R] = L;
}
```

提示 6-5：在双向链表这样的复杂链式结构中，往往会编写一些辅助函数用来设置链接关系。

有了这个代码，可以先记录好操作之前 X 和 Y 两边的结点，然后用 link 函数按照某种顺序把它们连起来。操作 4 比较特殊，为了避免一次修改所有元素的指针，此处增加一个标记 inv ，表示有没有执行过操作 4（如果 $\text{inv}=1$ 时再执行一次操作 4，则 inv 变为 0）。这样，当 op 为 1 和 2 且 $\text{inv}=1$ 时，只需把 op 变成 $3-\text{op}$ （注意操作 3 不受 inv 影响）即可。最终输出时要根据 inv 的值进行不同处理。

提示 6-6：如果数据结构上的某一个操作很耗时，有时可以用加标记的方式处理，而不需要真的执行那个操作。但同时，该数据结构的所有其他操作都要考虑这个标记。

下面的核心代码里还有一些可以借鉴的细节处理，请读者仔细阅读：

```

int main() {
    int m, kase = 0;
    while (scanf("%d%d", &n, &m) == 2) {
        for (int i = 1; i <= n; i++) {
            left[i] = i-1;
            right[i] = (i+1) % (n+1);
        }
        right[0] = 1; left[0] = n;
        int op, X, Y, inv = 0;

        while (m--) {
            scanf("%d", &op);
            if (op == 4) inv = !inv;
            else {
                scanf("%d%d", &X, &Y);
                if (op == 3 && right[Y] == X) swap(X, Y);
                if (op != 3 && inv) op = 3 - op;
                if (op == 1 && X == left[Y]) continue;
                if (op == 2 && X == right[Y]) continue;

                int LX = left[X], RX = right[X], LY = left[Y], RY = right[Y];
                if (op == 1) {
                    link(LX, RX); link(LY, X); link(X, Y);
                }
                else if (op == 2) {
                    link(LX, RX); link(Y, X); link(X, RY);
                }
                else if (op == 3) {
                    if (right[X] == Y) { link(LX, Y); link(Y, X); link(X, RY); }
                    else { link(LX, Y); link(Y, RX); link(LY, X); link(X, RY); }
                }
            }
        }

        int b = 0;
        long long ans = 0;
        for (int i = 1; i <= n; i++) {
            b = right[b];
            if (i % 2 == 1) ans += b;
        }
        if (inv && n % 2 == 0) ans = (long long)n*(n+1)/2 - ans;
        printf("Case %d: %lld\n", ++kase, ans);
    }
}

```



```
    }  
    return 0;  
}
```

如果读者曾独立编写过上面的程序，可能会花费较长的时间进行调试。又或者，自以为正确的程序提交到 UVa 上之后却得到 WA 甚至 RE 或者 TLE。在链式结构中，这样的情况是时常发生的，我们需要具备一定的调试和测试能力。

提示 6-7：复杂的链式数据结构往往比较容易写错。在包含多道题目的算法竞赛中，这一特点可以是选题的依据之一。

简单地说，测试的任务就是检查一份代码是否正确。如果找到了错误，最好还能提供一个让它错误的数据。有了错误数据之后，接下来的任务便是调试：看看程序为什么是错的。如果找到了错误，最好把它改对——至少对于刚才的错误数据能得到正确的结果。改对一组数据之后，可能还有其他错误，因此需要进一步测试；即使以前曾经正确的数据，也可能因为多次改动之后反而变错了，需要再次调试。总之，在编码结束后，为了确保程序的正确性，测试和调试往往要交替进行。

提示 6-8：测试的任务就是检查一份代码是否正确。如果找到了错误，最好还能提供一个让它出错的数据；调试的任务是找到错误原因并改正。改正一个错误之后有可能引入新的错误，因此调试和测试往往要交替进行。

如何测试上述代码的正确性呢？一个行之有效的方法是：再找一份完成同样功能的代码与之对比。对于本题来说，可以先写一个基于数组的版本。虽然这个版本会很慢，但正确性比较容易保证。接下来编写一个数据生成器（在第 5 章中曾介绍过这一技巧），并且反复执行下面的操作：生成随机数据，分别执行两个程序，比较它们的结果（俗称“对拍”）。合理地使用操作系统提供的脚本功能，可以自动完成对比测试，具体方法请读者参见附录 A。

提示 6-9：测试数据结构程序的常用方法是对拍：写一个功能相同但速度较慢的简易版本，再写一个数据生成器，不停对比快慢两个程序的输出。简易版本的代码越简单越好，因为重点不在效率，而在正确性。

如果发现让两个程序答案不一致的数据，最好别急着对它进行调试。可以尝试着减小数据生成器中的 n 和 m ，试图找到一组尽量简单的错误数据。一般来说，数据越简单，越容易调试。如果发现只有很大的数据才会出错，通常意味着程序在处理极限数据方面有问题，例如，`is_prime` 中遇到了“过大的 n ”，或者数组开得不够大等。这些都是很实用的技巧，建议读者多多积累。

提示 6-10：数据的复杂性会大大影响调试的难度，因此在找到让程序出错的数据之后最好别急着调试，而应尝试简化数据，或者直接用更小的参数调用数据生成器，以找到更简单的错误数据。

“对拍”也是命题者采用的常用技巧——为了保证官方测试数据的正确性，命题者通

常会请几个“验题者”编写程序。这些验题者往往还会故意编写错误或者速度较慢的程序，以确保这些程序会得到错误的结果，或者超时。对于一道算法竞赛的题目，正确性只是测试数据的最低要求，一套优秀的测试数据还要能全面地测出选手程序在正确性和效率上的缺陷，否则对辛辛苦苦写出正确程序的选手不公平。

6.3 树和二叉树

二叉树（Binary Tree）的递归定义如下：二叉树要么为空，要么由根结点（root）、左子树（left subtree）和右子树（right subtree）组成，而左子树和右子树分别是一棵二叉树。注意，在计算机中，树一般是“倒置”的，即根在上，叶子在下。

树（tree）和二叉树类似，区别在于每个结点不一定只有两棵子树。本书就是树状结构，根结点有 12 棵子树：第 1 章、第 2 章、第 3 章、……、第 12 章，而第 1 章又有 5 棵子树：1.1、1.2、……、1.5。

不管是二叉树还是树，每个非根结点都有一个父亲（father），也称父结点。

6.3.1 二叉树的编号

例题 6-6 小球下落（Dropping Balls, UVa 679）

有一棵二叉树，最大深度为 D ，且所有叶子的深度都相同。所有结点从上到下从左到右编号为 $1, 2, 3, \dots, 2^D - 1$ 。在结点 1 处放一个小球，它会往下落。每个内结点上都有一个开关，初始全部关闭，当每次有小球落到一个开关上时，状态都会改变。当小球到达一个内结点时，如果该结点上的开关关闭，则往左走，否则往右走，直到走到叶子结点，如图 6-2 所示。

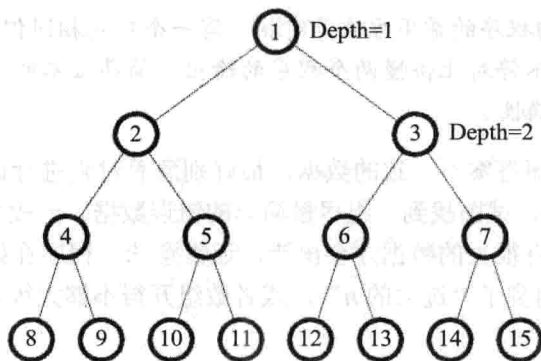


图 6-2 所有叶子深度相同的二叉树

一些小球从结点 1 处依次开始下落，最后一个小球将会落到哪里呢？输入叶子深度 D 和小球个数 I ，输出第 I 个小球最后所在的叶子编号。假设 I 不超过整棵树的叶子个数。 $D \leq 20$ 。输入最多包含 1000 组数据。

样例输入:

```
4 2
3 4
10 1
2 2
8 128
16 12345
```

样例输出:

```
12
7
512
3
255
36358
```

【分析】

不难发现, 对于一个结点 k , 其左子结点、右子结点的编号分别是 $2k$ 和 $2k+1$ 。这个结论非常重要, 请读者引起重视。

提示 6-11: 给定一棵包含 2^d 个结点 (其中 d 为树的高度) 的完全二叉树, 如果把结点从上到下从左到右编号为 $1, 2, 3, \dots$, 则结点 k 的左右子结点编号分别为 $2k$ 和 $2k+1$ 。

这样, 不难写出如下的模拟程序:

```
#include<cstdio>
#include<cstring>
const int maxd = 20;
int s[1<<maxd]; //最大结点数为  $2^{\max d}-1$ 
int main() {
    int D, I;
    while(scanf("%d%d", &D, &I) == 2) {
        memset(s, 0, sizeof(s)); //开关
        int k, n = (1<<D)-1; //n 是最大结点编号
        for(int i = 0; i < I; i++){ //连续让 I 个小球下落
            k = 1;
            for(;;) {
                s[k] = !s[k];
                k = s[k] ? k*2 : k*2+1; //根据开关状态选择下落方向
                if(k > n) break; //已经落“出界”了
            }
        }
        printf("%d\n", k/2); //“出界”之前的叶子编号
    }
}
```

```
return 0;
}
```

尽管在本题中，每个小球都是严格下落 $D-1$ 层，但用“if($k > n$) break”的方法判断“出界”更具一般性，所以上面的代码采用了这种方法。

这个程序和前面用数组模拟盒子移动的程序有一个共同的缺点：运算量太大。由于 I 可以高达 2^D-1 ，每组测试数据下落总层数可能会高达 $2^{19}-1=524287$ ，而一共可能有 10000 组数据……

每个小球都会落在根结点上，因此前两个小球必然是一个在左子树，一个在右子树。一般地，只需看小球编号的奇偶性，就能知道它是最终在哪棵子树中。对于那些落入根结点左子树的小球来说，只需知道该小球是第几个落在根的左子树里的，就可以知道它下一步往左还是往右了。依此类推，直到小球落到叶子上。

如果使用题目中给出的编号 I ，则当 I 是奇数时，它是往左走的第 $(I+1)/2$ 个小球；当 I 是偶数时，它是往右走的第 $I/2$ 个小球。这样，可以直接模拟最后一个小球的路线：

```
while(scanf("%d%d", &D, &I) == 2){
    int k = 1;
    for(int i = 0; i < D-1; i++){
        if(I%2) { k = k*2; I = (I+1)/2; }
        else { k = k*2+1; I /= 2; }
    }
    printf("%d\n", k);
}
```

这样，程序的运算量就与小球编号无关了，而且节省了一个巨大的 s 数组。

6.3.2 二叉树的层次遍历

例题 6-7 树的层次遍历 (Trees on the level, Duke 1993, UVa 122)

输入一棵二叉树，你的任务是按从上到下、从左到右的顺序输出各个结点的值。每个结点都按照从根结点到它的移动序列给出（L 表示左，R 表示右）。在输入中，每个结点的左括号和右括号之间没有空格，相邻结点之间用一个空格隔开。每棵树的输入用一对空括号“()”结束（这对括号本身不代表一个结点），如图 6-3 所示。

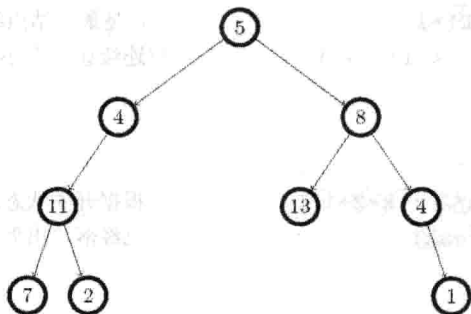


图 6-3 一棵二叉树

注意，如果从根到某个叶结点的路径上有的结点没有在输入中给出，或者给出超过一次，应当输出-1。结点个数不超过 256。

样例输入：

```
(11,LL) (7,LLL) (8,R) (5,) (4,L) (13,RL) (2,LLR) (1,RRR) (4,RR) ()
(3,L) (4,R) ()
```

样例输出：

```
5 4 8 11 13 4 7 2 1
```

```
-1
```

【分析】

受 6.3.1 节的启发，是否可以把树上的结点编号，然后把二叉树储存在数组中呢？很遗憾，这样的方法在本题中是行不通的。题目中已限制结点最多有 256 个。如果各个结点形成一条链，最后一个结点的编号将是巨大的！就算用高精度保存编号，数组也开不下。

看来，需要采用动态结构，根据需要建立新的结点，然后将其组织成一棵树。首先，编写输入部分和主程序：

```
char s[maxn]; //保存读入结点
bool read_input(){
    failed = false;
    root = newnode(); //创建根结点
    for(;;){
        if(scanf("%s", s) != 1) return false; //整个输入结束
        if(!strcmp(s, "()")) break; //读到结束标志，退出循环
        int v;
        sscanf(&s[1], "%d", &v); //读入结点的值
        addnode(v, strchr(s, ',')+1); //查找逗号，然后插入结点
    }
    return true;
}
```

程序不难理解：不停读入结点，如果在读到空括号之前文件结束，则返回 0（这样，在 main 函数里就能得知输入结束）。注意，这里两次用到了 C 语言中字符串的灵活性——可以把任意“指向字符的指针”看成是字符串，从该位置开始，直到字符“\0”。例如，若读到的结点是(11,LL)，则&s[1]所对应的字符串是“11,LL”。函数 strchr(s, ',')返回字符串 s 中从左往右第一个字符“,”的指针，因此 strchr(s, ',')+1 所对应的字符串是“LL”。这样，实际调用的是 addnode(11, "LL")。

接下来是重头戏了：二叉树的结点定义和操作。首先，需要定义一个称为 Node 的结构体，并且对应整棵二叉树的树根 root：

```
//结点类型
struct Node{
    bool have_value; //是否被赋值过
```

```
int v; //结点值
Node *left, *right;
Node():have_value(false),left(NULL),right(NULL){} //构造函数
};

Node* root; //二叉树的根结点
```

由于二叉树是递归定义的，其左右子结点类型都是“指向结点类型的指针”。换句话说，如果结点的类型为 `Node`，则左右子结点的类型都是 `Node*`。

提示 6-12：如果要定义一棵二叉树，一般是定义一个“结点”类型的 `struct`（如叫 `Node`），然后保存树根的指针（如 `Node* root`）。

每次需要一个新的 `Node` 时，都要用 `new` 运算符申请内存，并执行构造函数。下面把申请新结点的操作封装到 `newnode` 函数中：

```
Node* newnode() { return new Node(); }
```

提示 6-13：可以用 `new` 运算符申请空间并执行构造函数。如果返回值为 `NULL`，说明空间不足，申请失败。

接下来是在 `read_input` 中调用的 `addnode` 函数。它按照移动序列行走，目标不存在时调用 `newnode` 来创建新结点。

```
void addnode(int v, char* s){
    int n = strlen(s);
    Node* u = root; //从根结点开始往下走
    for(int i = 0; i < n; i++){
        if(s[i] == 'L'){
            if(u->left == NULL) u->left = newnode(); //结点不存在，建立新结点
            u = u->left; //往左走
        } else if(s[i] == 'R'){
            if(u->right == NULL) u->right = newnode();
            u = u->right;
        } //忽略其他情况，即最后那个多余的右括号
    }
    if(u->have_value) failed = true; //已经赋过值，表明输入有误
    u->v = v;
    u->have_value = true; //别忘记做标记
}
```

这样一来，输入和建树部分已经结束，接下来需要按照层次顺序遍历这棵树。此处使用一个队列来完成这个任务，初始时只有一个根结点，然后每次取出一个结点，就把它的左右子结点（如果存在）放进队列。

```
bool bfs(vector<int>& ans){
    queue<Node*> q;
```

```

ans.clear();
q.push(root); //初始时只有一个根结点
while(!q.empty()){
    Node* u = q.front(); q.pop();
    if(!u->have_value) return false; //有结点没有被赋值过，表明输入有误
    ans.push_back(u->v); //增加到输出序列尾部
    if(u->left != NULL) q.push(u->left); //把左子结点（如果有）放进队列
    if(u->right != NULL) q.push(u->right); //把右子结点（如果有）放进队列
}
return true; //输入正确
}

```

这样遍历二叉树的方法称为宽度优先遍历（Breadth-First Search, BFS）。后面将看到，BFS 在显示图和隐式图算法中扮演着重要的角色。

提示 6-14：可以用队列实现二叉树的层次遍历。这个方法还有一个名字，叫做宽度优先遍历（Breadth-First Search, BFS）。

上面的程序在功能上是正确的，但有一个小小的技术问题：在输入一组新数据时，没有释放上一棵二叉树所申请的内存空间。一旦执行了 `root = newnode()`，就再也无法访问到那些内存了，尽管那些内存物理上仍然存在。

当然，从技术上说，还是可以访问到那些内存的，如果能“猜到”那些地址。之所以说“访问不到”，是因为丢失了指向这些内存的指针。如果读者觉得这难以理解，想象一下丢失电话号码以后的情形：理论上仍然可以像以前一样给朋友打电话，只是没有了电话簿，查不到他们的号码了。

有一个专业术语用来描述这样的情况：内存泄漏（memory leak）——它意味着有些内存被白白浪费了。在实际运行的过程中，一般很难看出这个问题：在很多情况下，内存空间都不会很紧张，浪费一些空间后，程序还是可以正常运行；况且在整个程序结束后，该程序占用的空间会被操作系统全部回收，包括泄漏的那些。

提示 6-15：如果程序动态申请内存，请注意内存泄漏。程序执行完毕后，操作系统会回收该程序申请的所有内存（包括泄漏的），所以在算法竞赛中内存泄漏往往不会造成什么影响。但是，从专业素养的角度考虑，请从现在开始养成好习惯，对内存泄漏保持警惕。

下面是释放一棵二叉树的代码^①，请在“`root = newnode()`”之前加一行“`remove_tree(root)`”：

```

void remove_tree(Node* u) {
    if(u == NULL) return; //提前判断比较稳妥
    remove_tree(u->left); //递归释放左子树的空间
    remove_tree(u->right); //递归释放右子树的空间
    delete u; //调用 u 的析构函数并释放 u 结点本身的内存
}

```

^① 这样做虽然不会出现内存泄漏，但可能会出现内存碎片（memory fragmentation）。

二叉树并不一定要用指针实现。接下来,把指针完全去掉。首先还是给每个结点编号,但不是按照从上到下从左到右的顺序,而是按照结点的生成顺序。用计数器 `cnt` 表示已存在的结点编号的最大值,因此 `newnode` 函数需要改成这样:

```
const int root = 1;
void newtree() { left[root] = right[root] = 0; have_value[root] = false; cnt = root; }
int newnode() { int u = ++cnt; left[u] = right[u] = 0; have_value[root] = false; return u; }
```

上面的 `newtree()` 是用来代替前面的 “`remove_tree(root)`” 和 “`root = newnode()`” 两条语句的: 由于没有了动态内存的申请和释放,只需要重置结点计数器和根结点的左右子树了。

接下来,把所有的 `Node*` 类型改成 `int` 类型,然后把结点结构中的成员变量改成全局数组(例如, `u->left` 和 `u->right` 分别改成 `left[u]` 和 `right[u]`),除了 `char*` 外,整个程序就没有任何指针了。

提示 6-16: 可以用数组来实现二叉树,方法是用整数表示结点编号, `left[u]` 和 `right[u]` 分别表示 `u` 的左右子结点的编号。

虽然包括笔者在内的很多选手更喜欢用数组方式实现二叉树(因为编程简单,容易调试),但仍然需要具体问题具体分析。例如,用指针直接访问比 “数组+下标” 的方式略快,因此有的选手喜欢用 “结构体+指针” 的方式处理动态数据结构,但在申请结点时仍然用这里的 “动态化静态” 的思想,把 `newnode` 函数写成:

```
Node* newnode() { Node* u = &node[++cnt]; u->left = u->right = NULL; u->have_value = false; return u; }
```

其中, `node` 是静态申请的结构体数组。这样写的坏处在于 “释放内存” 很不方便(想一想,为什么)。如果反复执行新建结点和删除结点, `cnt` 会一直增加,但是用完的内存却无法重用。在大多数算法竞赛题目中,这并不会引起问题,但也有一些对内存要求极高的题目,对内存的一点浪费就会引起 “内存溢出” 错误。常见的解决方案是写一个简单的内存池 (memory pool), 具体来说就是维护一个空闲列表 (free list), 初始时把上述 `node` 数组中所有元素的指针放到该列表中,如下所示:

```
queue<Node*> freenodes;
Node node[maxn];

void init() {
    for(int i = 0; i < maxn; i++)
        freenodes.push(&node[i]); //初始化内存池
}

Node* newnode() {
```



```

Node* u = freenodes.front();
u->left = u->right = NULL; u->have_value = false; //重新初始化该结点
freenodes.pop();
return u;
}

```

```

void deletenode(Node* u) {

```

```

    freenodes.push(u);
}

```

提示 6-17: 可以用静态数组配合空闲列表来实现一个简单的内存池。虽然在大多数算法竞赛题目中用不上,但是内存池技术在高水平竞赛以及工程实践中都极为重要。

6.3.3 二叉树的递归遍历

对于二叉树 T , 可以递归定义它的先序遍历、中序遍历和后序遍历, 如下所示:

$\text{PreOrder}(T) = T \text{ 的根结点} + \text{PreOrder}(T \text{ 的左子树}) + \text{PreOrder}(T \text{ 的右子树})$

$\text{InOrder}(T) = \text{InOrder}(T \text{ 的左子树}) + T \text{ 的根结点} + \text{InOrder}(T \text{ 的右子树})$

$\text{PostOrder}(T) = \text{PostOrder}(T \text{ 的左子树}) + \text{PostOrder}(T \text{ 的右子树}) + T \text{ 的根结点}$

其中, 加号表示字符串连接运算。例如, 对于如图 6-4 所示的二叉树, 先序遍历为 DBACEGF, 中序遍历为 ABCDEFG。

这 3 种遍历都属于递归遍历, 或者说深度优先遍历 (Depth-First Search, DFS), 因为它总是优先往深处访问。

提示 6-18: 二叉树有 3 种深度优先遍历: 先序遍历、中序遍历和后序遍历。

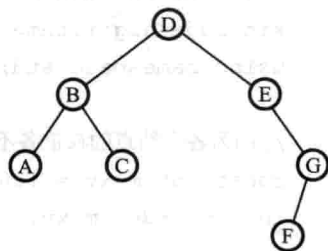


图 6-4 另一棵二叉树

例题 6-8 树 (Tree, UVa 548)

给一棵点带权 (权值各不相同, 都是小于 10000 的正整数) 的二叉树的中序和后序遍历, 找一个叶子使得它到根的路径上的权和最小。如果有多解, 该叶子本身的权应尽量小。输入中每两行表示一棵树, 其中第一行为中序遍历, 第二行为后序遍历。

样例输入:

```

3 2 1 4 5 7 6
3 1 2 5 6 7 4
7 8 11 3 5 16 12 18
8 3 11 7 16 18 12 5
255
255

```

样例输出：

```
1
3
255
```

【分析】

后序遍历的第一个字符就是根，因此只需在中序遍历中找到它，就知道左右子树的中序和后序遍历了。这样可以先把二叉树构造出来，然后再执行一次递归遍历，找到最优解。

提示 6-19：给定二叉树的中序遍历和后序遍历，可以构造出这棵二叉树。方法是根据后序遍历找到树根，然后在中序遍历中找到树根，从而找出左右子树的结点列表，然后递归构造左右子树。

代码如下：（另外，也可以在递归的同时统计最优解，不过程序稍微复杂一点，留给读者练习。）

```
#include<iostream>
#include<string>
#include<sstream>
#include<algorithm>
using namespace std;
```

//因为各个结点的权值各不相同且都是正整数，直接用权值作为结点编号

```
const int maxv = 10000 + 10;
```

```
int in_order[maxv], post_order[maxv], lch[maxv], rch[maxv];
```

```
int n;
```

```
bool read_list(int* a) {
```

```
    string line;
```

```
    if(!getline(cin, line)) return false;
```

```
    stringstream ss(line);
```

```
    n = 0;
```

```
    int x;
```

```
    while(ss >> x) a[n++] = x;
```

```
    return n > 0;
```

```
}
```

//把 in_order[L1..R1]和 post_order[L2..R2] 建成一棵二叉树，返回树根

```
int build(int L1, int R1, int L2, int R2) {
```

```
    if(L1 > R1) return 0; //空树
```

```
    int root = post_order[R2];
```

```
    int p = L1;
```

```

while(in_order[p] != root) p++;
int cnt = p-L1; //左子树的结点个数
lch[root] = build(L1, p-1, L2, L2+cnt-1);
rch[root] = build(p+1, R1, L2+cnt, R2-1);
return root;
}

int best, best_sum; //目前为止的最优解和对应的权和

void dfs(int u, int sum) {
    sum += u;
    if(!lch[u] && !rch[u]) { //叶子
        if(sum < best_sum || (sum == best_sum && u < best)) { best = u; best_sum
= sum; }
    }
    if(lch[u]) dfs(lch[u], sum);
    if(rch[u]) dfs(rch[u], sum);
}

int main() {
    while(read_list(in_order)) {
        read_list(post_order);
        build(0, n-1, 0, n-1);
        best_sum = 1000000000;
        dfs(post_order[n-1], 0);
        cout << best << "\n";
    }
    return 0;
}

```

例题 6-9 天平 (Not so Mobile, UVa 839)

输入一个树状天平，根据力矩相等原则判断是否平衡。如图 6-5 所示，所谓力矩相等，就是 $W_l D_l = W_r D_r$ ，其中 W_l 和 W_r 分别为左右两边砝码的重量， D 为距离。

采用递归（先序）方式输入：每个天平的格式为 W_l, D_l, W_r, D_r ，当 W_l 或 W_r 为 0 时，表示该“砝码”实际是一个子天平，接下来会描述这个子天平。当 $W_l = W_r = 0$ 时，会先描述左子天平，然后是右子天平。

样例输入：

1

0 2 0 4

0 3 0 1

```
1 1 1 1
2 4 4 2
1 6 3 2
```

其正确输出为 YES，对应图 6-6。

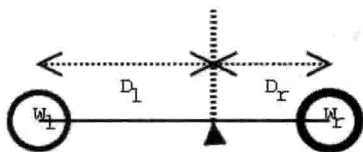


图 6-5 天平

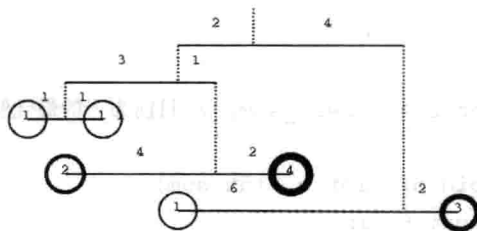


图 6-6 正确输出

【分析】

在解决这道题目之前，请先弄清楚题目的意思，尤其建议读者把样例输入画出来，以确保正确理解输入格式。

提示 6-20: 当题目比较复杂时，建议先手算样例或者至少把样例的图示画出来，以免误解题意。

这道题目的输入就采取了递归方式定义，因此编写一个递归过程进行输入比较自然。事实上，在输入过程中就能完成判断。由于使用引用传值，代码非常精简。

本题极为重要，请读者在继续阅读之前确保完全理解了下面的程序。

```
#include<iostream>
using namespace std;
//输入一个子天平，返回子天平是否平衡，参数 w 修改为子天平的总重量
bool solve(int& W) {
    int W1, D1, W2, D2;
    bool b1 = true, b2 = true;
    cin >> W1 >> D1 >> W2 >> D2;
    if(!W1) b1 = solve(W1);
    if(!W2) b2 = solve(W2);
    W = W1 + W2;
    return b1 && b2 && (W1 * D1 == W2 * D2);
}

int main() {
    int T, W;
    cin >> T;
    while(T--) {
        if(solve(W)) cout << "YES\n"; else cout << "NO\n";
        if(T) cout << "\n";
    }
}
```

```

    }
    return 0;
}

```

例题 6-10 下落的树叶 (The Falling Leaves, UVa 699)

给一棵二叉树，每个结点都有一个水平位置：左子结点在它左边 1 个单位，右子结点在右边 1 个单位。从左向右输出每个水平位置的所有结点的权值之和。如图 6-7 所示，从左到右的 3 个位置的权值和分别为 7，11，3。按照递归（先序）方式输入，用 -1 表示空树。

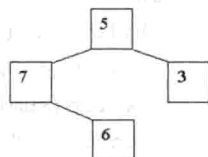


图 6-7 结点权值

样例输入：

```

5 7 -1 6 -1 -1 3 -1 -1
8 2 9 -1 -1 6 5 -1 -1 12 -1 -1 3 7 -1 -1 -1
-1

```

样例输出：

Case 1:

7 11 3

Case 2:

9 7 21 15

【分析】

本题和例题 6-9 很相似，但是实现细节比例题 6-9 略多，读者可以参考代码（这是一个不错的阅读练习）。为了节省篇幅，下面略去了唯一的全局变量 `int sum[maxn]`。

//输入并统计一棵子树，树根水平位置为 p

```

void build(int p) {
    int v; cin >> v;
    if(v == -1) return; //空树
    sum[p] += v;
    build(p - 1); build(p + 1);
}

```

//边读入边统计

```

bool init() {
    int v; cin >> v;
    if(v == -1) return false;
    memset(sum, 0, sizeof(sum));
    int pos = maxn/2; //树根的水平位置
    sum[pos] = v;
}

```

```

    build(pos - 1); build(pos + 1);
}

int main() {
    int kase = 0;
    while(init()) {
        int p = 0;
        while(sum[p] == 0) p++; //找最左边的叶子
        cout << "Case " << ++kase << ":\n" << sum[p++]; //因为要避免行末多余空格
        while(sum[p] != 0) cout << " " << sum[p++];
        cout << "\n\n";
    }
    return 0;
}

```

6.3.4 非二叉树

例题 6-11 四分树 (Quadrees, UVa 297)

如图 6-8 所示, 可以用四分树来表示一个黑白图像, 方法是用根结点表示整幅图像, 然后把行列各分成两等分, 按照图中的方式编号, 从左到右对应 4 个子结点。如果某子结点对应的区域全黑或者全白, 则直接用一个黑结点或者白结点表示; 如果既有黑又有白, 则用一个灰结点表示, 并且为这个区域递归建树。

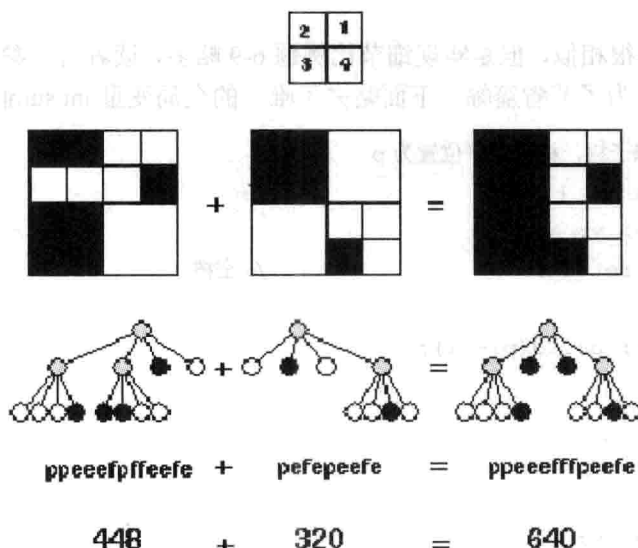


图 6-8 四分树

给出两棵四分树的先序遍历, 求二者合并之后 (黑色部分合并) 黑色像素的个数。p 表示中间结点, f 表示黑色 (full), e 表示白色 (empty)。

样例输入:

```
3
ppeeefpffeefe
pefepeeefe
peeef
peeefe
peeef
peeef
peepefefefe
```

样例输出:

```
There are 640 black pixels.
There are 512 black pixels.
There are 384 black pixels.
```

【分析】

由于四分树比较特殊,只需给出先序遍历就能确定整棵树(想一想,为什么)。只需要编写一个“画出来”的过程,边画边统计即可。

```
#include<cstdio>
#include<cstring>

const int len = 32;
const int maxn = 1024 + 10;
char s[maxn];
int buf[len][len], cnt;

//把字符串 s[p..] 导出到以 (r,c) 为左上角,边长为 w 的缓冲区中
//2 1
//3 4
void draw(const char* s, int& p, int r, int c, int w) {
    char ch = s[p++];
    if(ch == 'p') {
        draw(s, p, r, c+w/2, w/2); //1
        draw(s, p, r, c, w/2); //2
        draw(s, p, r+w/2, c, w/2); //3
        draw(s, p, r+w/2, c+w/2, w/2); //4
    } else if(ch == 'f') { //画黑像素(白像素不画)
        for(int i = r; i < r+w; i++)
            for(int j = c; j < c+w; j++)
                if(buf[i][j] == 0) { buf[i][j] = 1; cnt++; }
    }
}
```

```
int main() {
    int T;
    scanf("%d", &T);
    while(T--) {
        memset(buf, 0, sizeof(buf));
        cnt = 0;
        for(int i = 0; i < 2; i++) {
            scanf("%s", s);
            int p = 0;
            draw(s, p, 0, 0, len);
        }
        printf("There are %d black pixels.\n", cnt);
    }
    return 0;
}
```

6.4 图

图（Graph）描述的是一些个体之间的关系。与线性表和二叉树不同的是：这些个体之间既不是前驱后继的顺序关系，也不是祖先后代的层次关系，而是错综复杂的网状关系。

6.4.1 用 DFS 求连通块

例题 6-12 油田（Oil Deposits, UVa 572）

输入一个 m 行 n 列的字符矩阵，统计字符“@”组成多少个八连块。如果两个字符“@”所在的格子相邻（横、竖或者对角线方向），就说它们属于同一个八连块。例如，图 6-9 中有两个八连块。

```
****@
*@* *@
*@**@
@@* *@
@@**@
```

图 6-9 八连块

【分析】

和前面的二叉树遍历类似，图也有 DFS 和 BFS 遍历。由于 DFS 更容易编写，一般用 DFS 找连通块：从每个“@”格子出发，递归遍历它周围的“@”格子。每次访问一个格子时就给它写上一个“连通分量编号”（即下面代码中的 idx 数组），这样就可以在访问之前检查它是否已经有了编号，从而避免同一个格子访问多次：


```

#include<stdio>
#include<string>
const int maxn = 100 + 5;

char pic[maxn][maxn];
int m, n, idx[maxn][maxn];

void dfs(int r, int c, int id) {
    if(r < 0 || r >= m || c < 0 || c >= n) return; // “出界”的格子
    if(idx[r][c] > 0 || pic[r][c] != '@') return; //不是“@”或者已经访问过的格子
    idx[r][c] = id; //连通分量编号
    for(int dr = -1; dr <= 1; dr++)
        for(int dc = -1; dc <= 1; dc++)
            if(dr != 0 || dc != 0) dfs(r+dr, c+dc, id);
}

int main() {
    while(scanf("%d%d", &m, &n) == 2 && m && n) {
        for(int i = 0; i < m; i++) scanf("%s", pic[i]);
        memset(idx, 0, sizeof(idx));
        int cnt = 0;
        for(int i = 0; i < m; i++)
            for(int j = 0; j < n; j++)
                if(idx[i][j] == 0 && pic[i][j] == '@') dfs(i, j, ++cnt);
        printf("%d\n", cnt);
    }
    return 0;
}

```

上面的代码用一个二重循环来找到当前格子的相邻 8 个格子，也可以用常量数组或者写 8 条 DFS 调用，读者可以根据自己的喜好选用。这道题目的算法有个好听的名字：种子填充（floodfill）。有兴趣的读者还可以看看维基百科^①中的动画，对 DFS 和 BFS 实现的种子填充有一个更直观的认识。

提示 6-21：图也有 DFS 遍历和 BFS 遍历，其中前者用递归实现，后者用队列实现。求多维数组连通块的过程也称为种子填充（floodfill）。

例题 6-13 古代象形符号（Ancient Messages, World Finals 2011, UVa 1103）

本题的目的是识别 3000 年前古埃及用到的 6 种象形文字，如图 6-10 所示。

^① <http://en.wikipedia.org/wiki/Floodfill>.

					
Ankh	Wedjat	Djed	Scarab	Was	Akeht

图 6-10 古代象形符号

每组数据包含一个 H 行 W 列的字符矩阵 ($H \leq 200$, $W \leq 50$)，每个字符为 4 个相邻像素点的十六进制（例如，10011100 对应的字符就是 9c）。转化为二进制后 1 表示黑点，0 表示白点。输入满足：

- ❑ 不会出现上述 6 种符号之外的其他符号。
- ❑ 输入至少包含一个符号，且每个黑像素都属于一个符号。
- ❑ 每个符号都是一个四连块，并且不同符号不会相互接触，也不会相互包含。
- ❑ 如果两个黑像素有公共顶点，则它们一定有一个相同的相邻黑像素（有公共边）。
- ❑ 符号的形状一定和表 6-9 中的图形拓扑等价（可以随意拉伸但不能拉断）。

要求按照字典序输出所有符号。例如，图 6-11 中的输出应为 AKW。



图 6-11 输出 AKW

【分析】

“随意拉伸但不能拉断”是一个让人头疼的条件。怎么办呢？看来不能拘泥于细节，而要从全局考虑，找到一个易于计算，而且在“随意拉伸”时还不会改变的“特征量”，通过计算和比较“特征量”完成识别。题目说过，每个符号都是一个四连块，即所有黑点都连在一起，而中间有一些白色的“洞”。数一数就能发现，题目表中的 6 个符号从左到右依次有 1, 3, 5, 4, 0, 2 个洞，各不相同。这样，只需要数一数输入的符号有几个“白洞”，就能准确地知道它是哪个符号了。

6.4.2 用 BFS 求最短路

假设有一个网格迷宫，由 n 行 m 列的单元格组成，每个单元格要么是空地（用 1 来表示），要么是障碍物（用 0 来表示）。如何找到从起点到终点的最短路径？

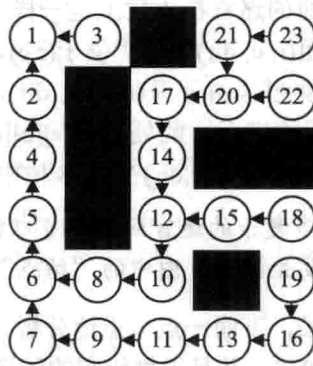
还记得二叉树的 BFS 吗？结点的访问顺序恰好是它们到根结点距离从小到大的顺序。类似地，也可以用 BFS 来按照到起点的距离顺序遍历迷宫图。

例如，假定起点在左上角，就从左上角开始用 BFS 遍历迷宫图，逐步计算出它到每个结点的最短路距离（如图 6-12 (a) 所示），以及这些最短路径上每个结点的“前一个结点”

(如图 6-12 (b) 所示)。

0	1		11	12
1		9	10	11
2		8		
3		7	8	9
4	5	6		10
5	6	7	8	9

(a) 从左上角出发到各个格子的最短距离



(b) 扩展顺序和父亲指针

图 6-12 用 BFS 求迷宫中最短路

注意,如果把图 6-12 (b) 中的箭头理解成“指向父亲的指针”,那么迷宫中的格子就变成了一棵树——除了起点之外,每个结点恰好有一个父亲。如果看不出来,可以把这棵树画成如图 6-13 所示的样子。这棵树称为最短路树,或者 BFS 树。

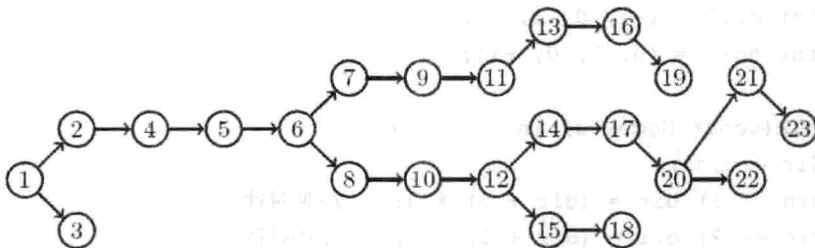


图 6-13 BFS 树的层次画法

例题 6-14 Abbott 的复仇 (Abbott's Revenge, ACM/ICPC World Finals 2000, UVa 816)

有一个最多包含 9×9 个交叉点的迷宫。输入起点、离开起点时的朝向和终点,求一条最短路(多解时任意输出一个即可)。

这个迷宫的特殊之处在于:进入一个交叉点的方向(用 NEWS 这 4 个字母分别表示北东西南,即上右下左)不同,允许出去的方向也不同。例如,1 2 WLF NR ER *表示交叉点(1,2)(上数第 1 行,左数第 2 列)有 3 个路标(字符“*”只是结束标志),如果进入该交叉点时的朝向为 W(即朝左),则可以左转(L)或者直行(F);如果进入时朝向为 N 或者 E 则只能右转(R),如图 6-14 所示。

注意:初始状态是“刚刚离开口”,所以即使出口和入口重合,最短路也不为空。例如,图 6-14 中的一条最短路为 (3,1) (2,1) (1,1) (1,2) (2,2) (2,3) (1,3) (1,2) (1,1) (2,1) (2,2) (1,2) (1,3) (2,3) (3,3)。

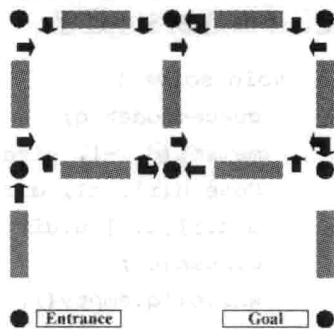


图 6-14 迷宫及走向

【分析】

本题和普通的迷宫在本质上是一样的,但是由于“朝向”也起到了关键作用,所以需要用一个三元组 (r, c, dir) 表示“位于 (r, c) , 面朝 dir ”这个状态。假设入口位置为 (r_0, c_0) , 朝向为 dir , 则初始状态并不是 (r_0, c_0, dir) , 而是 (r_1, c_1, dir) , 其中, (r_1, c_1) 是 (r_0, c_0) 沿着方向 dir 走一步之后的坐标。此处用 $d[r][c][\text{dir}]$ 表示初始状态到 (r, c, dir) 的最短路长度, 并且用 $p[r][c][\text{dir}]$ 保存了状态 (r, c, dir) 在 BFS 树中的父结点。

提示 6-22: 很多复杂的迷宫问题都可以转化为最短路问题, 然后用 BFS 求解。在套用 BFS 框架之前, 需要先搞清楚图中的“结点”包含哪些内容。

代码比较长, 下面一点一点地分析。首先是输入过程。将 4 个方向和 3 种“转弯方式”编号为 0~3 和 0~2, 并且提供相应的转换函数:

```
const char* dirs = "NESW";           //顺时针旋转
const char* turns = "FLR";
int dir_id(char c) { return strchr(dirs, c) - dirs; }
int turn_id(char c) { return strchr(turns, c) - turns; }
```

接下来是“行走”函数, 根据当前状态和转弯方式, 计算出后继状态:

```
const int dr[] = {-1, 0, 1, 0};
const int dc[] = {0, 1, 0, -1};

Node walk(const Node& u, int turn) {
    int dir = u.dir;
    if(turn == 1) dir = (dir + 3) % 4;    //逆时针
    if(turn == 2) dir = (dir + 1) % 4;    //顺时针
    return Node(u.r + dr[dir], u.c + dc[dir], dir);
}
```

输入函数比较简单, 作用就是读取 r_0, c_0, dir , 并且计算出 r_1, c_1 , 然后读入 has_edge 数组, 其中 $\text{has_edge}[r][c][\text{dir}][\text{turn}]$ 表示当前状态是 (r, c, dir) , 是否可以沿着转弯方向 turn 行走。下面是 BFS 主过程:

```
void solve() {
    queue<Node> q;
    memset(d, -1, sizeof(d));
    Node u(r1, c1, dir);
    d[u.r][u.c][u.dir] = 0;
    q.push(u);
    while(!q.empty()) {
        Node u = q.front(); q.pop();
        if(u.r == r2 && u.c == c2) { print_ans(u); return; }
        for(int i = 0; i < 3; i++) {
            Node v = walk(u, i);
```

```

    if(has_edge[u.r][u.c][u.dir][i] && inside(v.r, v.c)
    && d[v.r][v.c][v.dir] < 0) {
        d[v.r][v.c][v.dir] = d[u.r][u.c][u.dir] + 1;
        p[v.r][v.c][v.dir] = u;
        q.push(v);
    }
}
printf("No Solution Possible\n");
}

```

最后是解的打印过程。它也可以写成递归函数，不过用 `vector` 保存结点可以避免递归时出现栈溢出，并且更加灵活。

提示 6-23: 使用 BFS 求出图的最短路之后，可以用递归方式打印最短路的具体路径。如果最短路非常长，递归可能会引起栈溢出，此时可以改用循环，用 `vector` 保存路径。

```

void print_ans(Node u) {
    //从目标结点逆序追溯到初始结点
    vector<Node> nodes;
    for(;;) {
        nodes.push_back(u);
        if(d[u.r][u.c][u.dir] == 0) break;
        u = p[u.r][u.c][u.dir];
    }
    nodes.push_back(Node(r0, c0, dir));

    //打印解，每行 10 个
    int cnt = 0;
    for(int i = nodes.size()-1; i >= 0; i--) {
        if(cnt % 10 == 0) printf(" ");
        printf(" (%d,%d)", nodes[i].r, nodes[i].c);
        if(++cnt % 10 == 0) printf("\n");
    }
    if(nodes.size() % 10 != 0) printf("\n");
}

```

本题非常重要，强烈建议读者搞懂所有细节，并能独立编写程序。

6.4.3 拓扑排序

例题 6-15 给任务排序 (Ordering Tasks, UVa 10305)

假设有 n 个变量，还有 m 个二元组 (u, v) ，分别表示变量 u 小于 v 。那么，所有变量从小到大排列起来应该是什么样子的呢？例如，有 4 个变量 a, b, c, d ，若已知 $a < b, c < b, d < c$ ，

则这4个变量的排序可能是 $a < d < c < b$ 。尽管还有其他可能（如 $d < a < c < b$ ），你只需找出其中一个即可。

【分析】

把每个变量看成一个点，“小于”关系看成有向边，则得到了一个有向图。这样，我们的任务实际上是把一个图的所有结点排序，使得每一条有向边 (u, v) 对应的 u 都排在 v 的前面。在图论中，这个问题称为拓扑排序（topological sort）。

不难发现：如果图中存在有向环，则不存在拓扑排序，反之则存在。不包含有向环的有向图称为有向无环图（Directed Acyclic Graph, DAG）。可以借助 DFS 完成拓扑排序：在访问完一个结点之后把它加到当前拓扑序的首部（想一想，为什么不是尾部）。

```
int c[maxn];
int topo[maxn], t;
bool dfs(int u){
    c[u] = -1; //访问标志
    for(int v = 0; v < n; v++) if(G[u][v]) {
        if(c[v]<0) return false; //存在有向环，失败退出
        else if(!c[v] && !dfs(v)) return false;
    }
    c[u] = 1; topo[--t]=u;
    return true;
}
bool toposort(){
    t = n;
    memset(c, 0, sizeof(c));
    for(int u = 0; u < n; u++) if(!c[u])
        if(!dfs(u)) return false;
    return true;
}
```

这里用到了一个 c 数组， $c[u]=0$ 表示从来没有访问过（从来没有调用过 $\text{dfs}(u)$ ）； $c[u]=1$ 表示已经访问过，并且还递归访问过它的所有子孙（即 $\text{dfs}(u)$ 曾被调用过，并已返回）； $c[u]=-1$ 表示正在访问（即递归调用 $\text{dfs}(u)$ 正在栈帧中，尚未返回）。

提示 6-24：可以用 DFS 求出有向无环图（DAG）的拓扑排序。如果排序失败，说明该有向图存在有向环，不是 DAG。

6.4.4 欧拉回路

有一条名为 Pregel 的河流经过 Königsberg 城。城中有 7 座桥，把河中的两个岛与河岸连接起来。当地居民热衷于一个难题：是否存在一条路线，可以不重复地走遍 7 座桥。这就是著名的七桥问题。它由大数学家欧拉首先提出，并给出了完美的解答，如图 6-15 所示。

欧拉首先把图 6-15（a）中的七桥问题用图论的语言改写成图 6-15（b），则问题变成

了：能否从无向图中的一个结点出发走出一条道路，每条边恰好经过一次。这样的路线称为欧拉道路（eulerian path），也可以形象地称为“一笔画”。

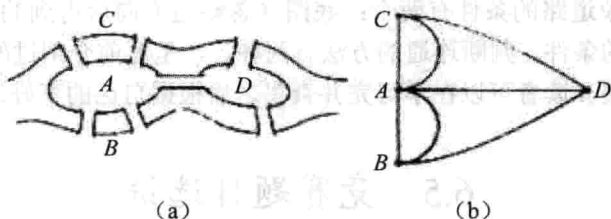


图 6-15 七桥问题

不难发现，在欧拉道路中，“进”和“出”是对应的——除了起点和终点外，其他点的“进出”次数应该相等。换句话说，除了起点和终点外，其他点的度数（degree）应该是偶数。很可惜，在七桥问题中，所有 4 个点的度数均是奇数（这样的点也称奇点），因此不可能存在欧拉道路。上述条件也是充分条件——如果一个无向图是连通的，且最多只有两个奇点，则一定存在欧拉道路。如果有两个奇点，则必须从其中一个奇点出发，另一个奇点终止；如果奇点不存在，则可以从任意点出发，最终一定会回到该点（称为欧拉回路）。

用类似的推理方式可以得到有向图的结论：最多只能有两个点的入度不等于出度，而且必须是其中一个点的出度恰好比入度大 1（把它作为起点），另一个的入度比出度大 1（把它作为终点）。当然，还有一个前提条件：在忽略边的方向后，图必须是连通的。

下面是程序，它同时适用于欧拉道路和回路。但如果需要打印的是欧拉道路，在主程序中调用时，参数必须是道路的起点。另外，打印的顺序是逆序的，因此在真正使用这份代码时，应当把 printf 语句替换成一条 push 语句，把边(u,v)压入一个栈内。

```
void euler(int u){
    for(int v = 0; v < n; v++) if(G[u][v] && !vis[u][v]) {
        vis[u][v] = vis[v][u] = 1;
        euler(v);
        printf("%d %d\n", u, v);
    }
}
```

尽管上面的代码只适用于无向图，但不难改成有向图：把 $\text{vis}[u][v] = \text{vis}[v][u] = 1$ 改成 $\text{vis}[u][v]$ 即可。

提示 6-25：根据连通性和度数可以判断出无向图和有向图是否存在欧拉道路和欧拉回路。可以用 DFS 构造欧拉回路和欧拉道路。

例题 6-16 单词（Play On Words, UVa 10129）

输入 n ($n \leq 100000$) 个单词，是否可以把所有这些单词排成一个序列，使得每个单词的第一个字母和上一个单词的最后一个字母相同（例如 acm、malform、mouse）。每个单词最多包含 1000 个小写字母。输入中可以有重复单词。

【分析】

把字母看作结点，单词看成有向边，则问题有解，当且仅当图中有欧拉路径。前面讲过，有向图存在欧拉道路的条件有两个：底图（忽略边方向后得到的无向图）连通，且度数满足上面讨论过的条件。判断连通的方法有两种，一是之前介绍过的 DFS，二是第 11 章中将要介绍的并查集。读者可以在学习完并查集之后根据自己的喜好选用。

6.5 竞赛题目选讲

例题 6-17 看图写树（Undraw the Trees, UVa 10562）

你的任务是将多叉树转化为括号表示法。如图 6-16 所示，每个结点用除了“-”、“|”和空格的其他字符表示，每个非叶结点的正下方总会有一个“|”字符，然后下方是一排“-”字符，恰好覆盖所有子结点的上方。单独的一行“#”为数据结束标记。

样例输入	样例输出
<pre> 2 A --- B C D --- E F G # e --- f g # </pre>	<pre> (A(B()C(E()F()))D(G())) (e(f()g())) </pre>

图 6-16 样例输入与输出

【分析】

直接在二维字符数组里递归即可，无须建树。注意对空树的处理，以及结点标号可以是任意可打印字符。代码如下：

```

#include<cstdio>
#include<cctype>
#include<cstring>
using namespace std;

const int maxn = 200 + 10;
int n;
char buf[maxn][maxn];

//递归遍历并且输出以字符 buf[r][c] 为根的树
void dfs(int r, int c) {
    printf("%c(", buf[r][c]);

```



```

if(r+1 < n && buf[r+1][c] == '|') { //有子树
    int i = c;
    while(i-1 >= 0 && buf[r+2][i-1] == '-') i--; //找“----”的左边界
    while(buf[r+2][i] == '-' && buf[r+3][i] != '\0') {
        if(!isspace(buf[r+3][i])) dfs(r+3, i); //fgets 读入的“\n”也满足 isspace()
        i++;
    }
}
printf(" ");
}

void solve() {
    n = 0;
    for(;;) {
        fgets(buf[n], maxn, stdin);
        if(buf[n][0] == '#') break; else n++;
    }
    printf("(");
    if(n) {
        for(int i = 0; i < strlen(buf[0]); i++)
            if(buf[0][i] != ' ') { dfs(0, i); break; }
    }
    printf("\n");
}

int main() {
    int T;
    fgets(buf[0], maxn, stdin);
    sscanf(buf[0], "%d", &T);
    while(T--) solve();
    return 0;
}

```

例题 6-18 雕塑 (Sculpture, ACM/ICPC NWERC 2008, UVa12171)

某雕塑由 n ($n \leq 50$) 个边平行于坐标轴的长方体组成。每个长方体用 6 个整数 x_0, y_0, z_0, x, y, z 表示 (均为 1~500 的整数), 其中 x_0 为长方体的顶点中 x 坐标的最小值, x 表示长方体在 x 方向的总长度。其他 4 个值类似定义。你的任务是统计这个雕像的体积和表面积。注意, 雕塑内部可能会有密闭的空间, 其体积应计算在总体积中, 但从“外部”看不见的面不应计入表面积。雕塑可能会由多个连通块组成。

【分析】

设想有一个三维坐标范围均为 1~500 个三维网格, 如果一开始就把输入的 n 个长方体“画”到网格里, 接下来就可以抛开那些长方体, 只在网格中进行统计了。

还记得 floodfill 吗？它不仅能求出连通块的个数，还能准确地找出每个连通块各由哪些方格组成。虽然本题的研究对象是三维空间中的长方体，但丝毫不影响 floodfill 的作用，唯一的区别就是每个格子的相邻格子从二维情形的 4 个增加到了三维情形的 8 个。

本题的麻烦之处在于雕塑中间可能有封闭区域，甚至还有可能相互嵌套，看上去很复杂。但其实可以从反面思考：不考虑雕塑本身，而考虑“空气”。在网格周围加一圈“空气”（目的是为了让所有空气格子连通），然后做一次 floodfill，就可以得到空气的“内表面积”和体积。这个表面积就是雕塑的外表面积，而雕塑体积等于总体积减去空气体积。

但还有一个大问题：空间占用。坐标为 1~500 的整数，一共需要 $500^3=1.25 \times 10^8$ 个单元。在第 5 章的例题“城市正视图”中介绍了离散化法，在这里它再次派上用场：每个维度最多只有 $2n \leq 100$ 个不同的坐标，因此可以把 $500 \times 500 \times 500$ 的网格离散化成 $100 \times 100 \times 100$ ，单元格的数目降为原来的 1/125。在 floodfill 时直接使用离散化后的新网格，但在统计表面积和体积时则需要使用原始坐标。

例题 6-19 自组合 (Self-Assembly, ACM/ICPC World Finals 2013, UVa 1572)

有 n ($n \leq 40000$) 种边上带标号的正方形。每条边上的标号要么为一个写字母后面跟着一个加号或减号，要么为数字 00。当且仅当两条边的字母相同且符号相反时，两条边能拼在一起（00 不能和任何边拼在一起，包括另一条标号为 00 的边）。

假设输入的每种正方形都有无穷多种，而且可以旋转和翻转，你的任务是判断能否组成一个无限大的结构。每条边要么悬空（不和其他边相邻），要么和一个上述可拼接的边相邻。如图 6-17 (a) 所示是 3 个正方形，图 6-17 (b) 所示边是它们组成的一个合法结构（但大小有限）。

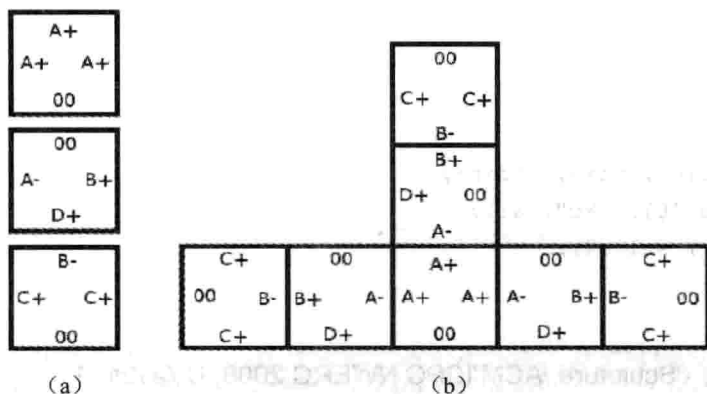


图 6-17 自组合正方形

【分析】

本题看上去很难下手，但不难发现“可以旋转和翻转”是一个很有意思的条件，值得推敲。“无限大结构”并不一定能铺满整个平面，只需要能连出一条无限长的“通路”即可。借助于旋转和翻转，可以让这条“通路”总是往右和往下延伸，因此永远不会自交。这样一来，只需以某个正方形为起点开始“铺路”，一旦可以拼上一块和起点一样的正方形，无限重复下去就能得到一个无限大的结构。

可惜这样的分析仍然不够，因为正方形的数目 n 很大。进一步分析发现：实际上不需要正方形本身重复，而只需要边上的标号重复即可。这样问题就转化为：把标号看成点（一共只有 $A^+ \sim Z^+$, $A^- \sim Z^-$ 这 52 种，因为 00 不能作为拼接点），正方形看作边，得到一个有向图。则当且仅当图中存在有向环时有解。只需要做一次拓扑排序即可。

例题 6-20 理想路径 (Ideal Path, NEERC 2010, UVa1599)

给一个 n 个点 m 条边 ($2 \leq n \leq 100000$, $1 \leq m \leq 200000$) 的无向图，每条边上都涂有一种颜色。求从结点 1 到结点 n 的一条路径，使得经过的边数尽量少，在此前提下，经过边的颜色序列的字典序最小。一对结点间可能有多条边，一条边可能连接两个相同结点。输入保证结点 1 可以达到结点 n 。颜色为 $1 \sim 10^9$ 的整数。

【分析】

首先回顾一下第 3 章中介绍的“字典序”。对于字符串来说，字典序就是在字典里的顺序。例如，ab 在 cd 的前面，cde 在 a 的后面，abcd 在 abcde 的前面。这个定义可以扩展到序列：序列 (1, 2) 在 (3, 4, 5) 的前面，(4, 5, 6) 在 (4, 5) 的后面。

抛开字典序不谈，本题只是一个普通的最短路问题，可以用 BFS 解决。但是之前的“记录父结点”的方法已经不适用了，因为这样打印出来的路径并不能保证字典序最小。怎么办呢？

事实上，无须记录父结点也能得到最短路，方法是从终点开始“倒着”BFS，得到每个结点 i 到终点的最短距离 $d[i]$ ，然后直接从起点开始走，但是每次到达一个新结点时要保证 d 值恰好减少 1（如有多个选择则可以随便走），直到到达终点。可以证明（想一想，为什么）：这样走过的路径一定是一条最短路。

有了上述结论，本题就不难解决了：直接从起点开始按照上述规则走，如果有多种走法，选颜色字典序最小的走；如果有多条边的颜色字典序都是最小，则记录所有这些边的终点，走下一步时要考虑从所有这些点出发的边。聪明的读者应该已经看出来了：这实际上是又做了一次 BFS，因此时间复杂度仍为 $O(m)$ 。其实本题也可以只进行一次 BFS，不过要从终点开始逆向进行，有兴趣的读者可以自行研究。

本题非常重要，强烈建议读者编写程序。

例题 6-21 系统依赖 (System Dependencies, ACM/ICPC World Finals 1997, UVa506)

软件组件之间可能会有依赖关系，例如，TELNET 和 FTP 都依赖于 TCP/IP。你的任务是模拟安装和卸载软件组件的过程。首先是一些 DEPEND 指令，说明软件之间的依赖关系（保证不存在循环依赖），然后是一些 INSTALL、REMOVE 和 LIST 指令，如表 6-1 所示。

表 6-1 指令说明

指 令	说 明
DEPEND item1 item2 [item3 ...]	item1 依赖组件 item2, item3, ...
INSTALL item1	安装 item1 和它的依赖（已安装过的不用重新安装）
REMOVE item1	卸载 item1 和它的依赖（如果某组件还被其他显式安装的组件所依赖，则不能卸载这个组件）
LIST	输出所有已安装组件

在 INSTALL 指令中提到的组件称为显式安装,这些组件必须用 REMOVE 指令显式删除。同样地,被这些显式安装组件所直接或间接依赖的其他组件也不能在 REMOVE 指令中删除。

每行指令包含不超过 80 个字符,所有组件名称都是大小写敏感的。指令名称均为大写字母。

【分析】

这道题目在概念上并没有什么难点,但是有一些细节问题容易写错。首先,维护一个组件名字列表,这样可以把输入中的组件名全部转化为整数编号。接下来用两个 vector 数组 depend[x]和 depend2[x]分别表示组件 x 所依赖的组件列表和依赖于 x 的组件列表(即当读到 DEPEND x y 时要把 y 加入 depend[x],把 x 加入 depend2[y]),这样就可以方便地安装、删除组件,以及判断某个组件是否仍然需要了。

为了区分显式安装和隐式,需要一个数组 status[x],0 表示组件 x 未安装,1 表示隐式显式安装,2 表示隐式安装,则安装组件的代码如下:

```
void install(int item, bool toplevel) {
    if(!status[item]) {
        for(int i = 0; i < depend[item].size(); i++)
            install(depend[item][i], false);
        cout << "    Installing " << name[item] << "\n";
        status[item] = toplevel ? 1 : 2;
        installed.push_back(item);
    }
}
```

删除的顺序相反:首先判断本组件是否能删除,如果可以删除,在删除之后再递归删除它所依赖的组件:

```
bool needed(int item) {
    for(int i = 0; i < depend2[item].size(); i++)
        if(status[depend2[item][i]]) return true;
    return false;
}

void remove(int item, bool toplevel) {
    if((toplevel || status[item] == 2) && !needed(item)) {
        status[item] = 0;
        installed.erase(remove(installed.begin(), installed.end(), item),
                        installed.end());
        cout << "    Removing " << name[item] << "\n";
        for(int i = 0; i < depend[item].size(); i++)
            remove(depend[item][i], false);
    }
}
```

例题 6-22 战场 (Paintball, UVa 11853)

有一个 1000×1000 的正方形战场, 战场西南角的坐标为 $(0,0)$, 西北角的坐标为 $(0,1000)$ 。战场上有 n ($0 \leq n \leq 1000$) 个敌人, 第 i 个敌人的坐标为 (x_i, y_i) , 攻击范围为 r_i 。为了避免敌人的攻击, 在任意时刻, 你与每个敌人的距离都必须严格大于它的攻击范围。你的任务是从战场的西边 ($x=0$ 的某个点) 进入, 东边 ($x=1000$ 的某个点) 离开。如果有多个位置可以进/出, 你应当求出最靠北的位置。输入每个敌人的 x_i 、 y_i 、 r_i , 输出进入战场和离开战场的坐标。

【分析】

本题初看起来比较麻烦, 不妨把它简化一下: 先判断是否有解, 再考虑如何求出最靠北的位置。首先, 可以把每个敌人抽象成一个圆, 圆心就是他所在位置, 半径是攻击范围, 则本题变成了: 正方形内有 n 个圆形障碍物, 是否能从左边界走到右边界?

下一步需要一点创造性思维: 把正方形战场看成一个湖, 障碍物看成踏脚石, 如果可以从上边界“走”到下边界, 沿途经过的障碍物就会把湖隔成左右两半, 相互无法到达, 即本题无解; 另一方面, 如果从上边界走不到下边界, 虽然仍然可能会出现某些封闭区域 (图 6-18 中灰色区域), 但一定可以从左边界的某个地方到达右边界的某个地方, 如图 6-18 所示。

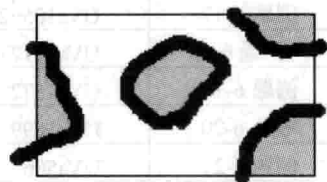


图 6-18 战场示意图

这样, 解的存在性只需一次 DFS 或 BFS 判连通即可。如何求出最北的进/出位置呢? 方法如下: 从上边界开始遍历, 沿途检查与边界相交的圆。这些圆和左边界的交点中最靠南边的一个就是所求的最北进入位置, 和右边界的最南交点就是所求的最北离开位置。

6.6 训练参考

本章介绍形形色色的数据结构, 包括线性表、树状结构和图。其中线性表的很多实现技巧已经在第 5 章中讨论过, 但是树和图的内容是全新的。树及其遍历是初学者学习数据结构的一个门槛, 所以本章展示了很多代码。本章中介绍的“图”仅是基本概念和最常用的算法, 但仍有不少问题仅需要这些概念和基本算法就能解决, 建议读者仔细体会本章的竞赛题目。

表 6-2 为例题列表, 其中带星号的是难度较大的题目。

表 6-2 例题列表

类 别	题 号	题目名称 (英文)	备 注
例题 6-1	UVa210	Concurrency Simulator	双端队列
例题 6-2	UVa514	Rails	栈
例题 6-3	UVa442	Matrix Chain Multiplication	用栈实现简单的表达式解析
例题 6-4	UVa11988	Broken Keyboard (a.k.a. Beiju Text)	链表
例题 6-5	UVa12657	Boxes in a Line	双向链表

续表

类 别	题 号	题目名称（英文）	备 注
例题 6-6	UVa679	Dropping Balls	完全二叉树编号
例题 6-7	UVa122	Trees on the level	二叉树的动态创建与 BFS
例题 6-8	UVa548	Tree	从中序和后序恢复二叉树
例题 6-9	UVa839	Not so Mobile	二叉树的 DFS
例题 6-10	UVa699	The Falling Leaves	二叉树的 DFS
例题 6-11	UVa297	Quadrees	四分树
例题 6-12	UVa572	Oil Deposits	图的连通块（DFS）
例题 6-13	UVa1103	Ancient Messages	图的连通块的应用
例题 6-14	UVa816	Abbott's Revenge	图的最短路（BFS）
例题 6-15	UVa10305	Ordering Tasks	拓扑排序
例题 6-16	UVa10129	Play On Words	欧拉回路
例题 6-17	UVa10562	Undraw the Trees	多叉树的 DFS
*例题 6-18	UVa12171	Sculpture	离散化; floodfill
例题 6-19	UVa1572	Self-Assembly	图论模型
例题 6-20	UVa1599	Ideal Path	图的 BFS 树
例题 6-21	UVa506	System Dependencies	图的概念和拓扑序
*例题 6-22	UVa11853	Paintball	对偶图

接下来是习题。本章的习题大都很传统，但部分题目的意思比较复杂，需要认真理解。建议读者完成至少 8 道习题，最好是 10 道以上。

习题 6-1 平衡的括号（Parentheses Balance, UVa 673）

输入一个包含“()”和“[]”的括号序列，判断是否合法。具体规则如下：

- 空串合法。
- 如果 A 和 B 都合法，则 AB 合法。
- 如果 A 合法则(A)和[A]都合法。

习题 6-2 S 树（S-Trees, UVa 712）

给出一棵满二叉树，每一层代表一个 01 变量，取 0 时往左走，取 1 时往右走。例如，图 6-19（a）和图 6-19（b）都对应表达式 $x_1 \wedge (x_2 \vee x_3)$ 。

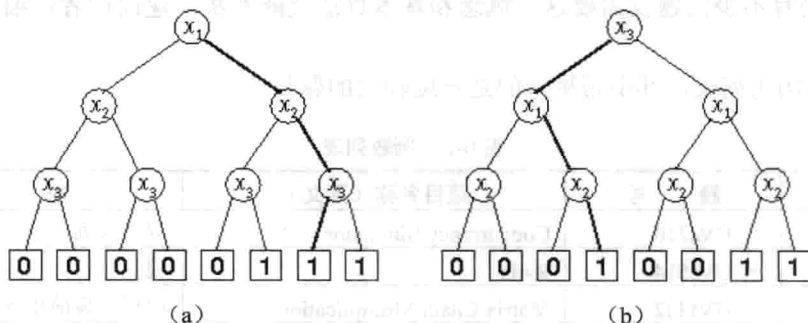


图 6-19 S 树

给出所有叶子的值以及一些查询（即每个变量 x_i 的取值），求每个查询到达的叶子的值。例如，有 4 个查询：000、010、111、110，则输出应为 0011。

习题 6-3 二叉树重建 (Tree Recovery, ULM 1997, UVa 536)

输入一棵二叉树的先序遍历和中序遍历序列，输出后序遍历序列，如图 6-20 所示。

样例输入	样例输出
DBACEGF ABCDEFG	ACBFGED
BCAD CBAD	CDAB

图 6-20 二叉树重建

习题 6-4 骑士的移动 (Knight Moves, UVa 439)

输入标准 8*8 国际象棋棋盘上的两个格子（列用 a~h 表示，行用 1~8 表示），求马最少需要多少步从起点跳到终点。例如从 a1 到 b2 需要 4 步。马的移动方式如图 6-21 所示。

习题 6-5 巡逻机器人 (Patrol Robot, ACM/ICPC Hanoi 2006, UVa1600)

机器人要从一个 $m*n$ ($1 \leq m, n \leq 20$) 网格的左上角(1,1)走到右下角(m,n)。网格中的一些格子是空地（用 0 表示），其他格子是障碍（用 1 表示）。机器人每次可以往 4 个方向走一格，但不能连续地穿越 k ($0 \leq k \leq 20$) 个障碍，求最短路径长度。起点和终点保证是空地。例如，对于图 6-22 (a) 中的数据，图 6-22 (b) 中显示的是最优解，路径长度为 10。

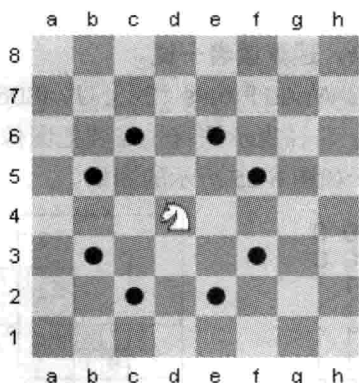


图 6-21 马的移动方式

```

0 1 1 0 0 0
0 0 1 0 1 1
0 1 1 1 1 0
0 1 1 1 0 0

```

(a)

```

0 1 1 0 0 0
0 0 1 0 1 1
0 1 1 1 1 0
0 1 1 1 0 0

```

(b)

图 6-22 最短路径示例

习题 6-6 修改天平 (Equilibrium Mobile, NWERC 2008, UVa12166)

给一个深度不超过 16 的二叉树，代表一个天平。每根杆都悬挂在中间，每个秤砣的重量已知。至少修改多少个秤砣的重量才能让天平平衡？如图 6-23 所示，把 7 改成 3 即可。

习题 6-7 Petri 网模拟 (Petri Net Simulation, ACM/ICPC World Finals 1998, UVa804)

你的任务是模拟 Petri 网的变迁。Petri 网包含 NP 个库所（用 P1, P2...表示）和 NT 个变迁（用 T1, T2...表示）。 $0 < NP, NT < 100$ 。当每个变迁的每个输入库所都至少有一个 token 时，变迁是允许的。变迁发生的结果是每个输入库所减少一个 token，每个输出库所增加一个 token。变迁的发生是原子性的，即所有 token 的增加和减少应同时进行。注意，一个变迁可能有多个相同的输入或者输出。如果一个库所在变迁的输入库所列表中出现了两次，

则 token 会减少两个。输出库所也是类似。如果有多个变迁是允许的，一次只能发生一个。

如图 6-24 所示，一开始只有 T1 是允许的，发生一次 T1 变迁之后有一个 token 会从 P1 移动到 P2，但仍然只有 T1 是允许的，因为 T2 要求 P2 有两个 token。再发生一次 T1 变迁之后 P1 中只剩一个 token，而 P2 中有两个，因为 T1 和 T2 都可以发生。假定 T2 发生，则 P2 中不再有 token，而 P3 中有一个 token，因此 T1 和 T3 都是允许的。

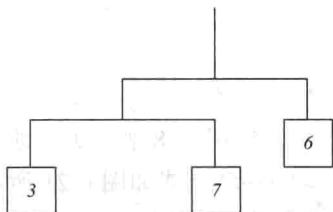


图 6-23 修改天平

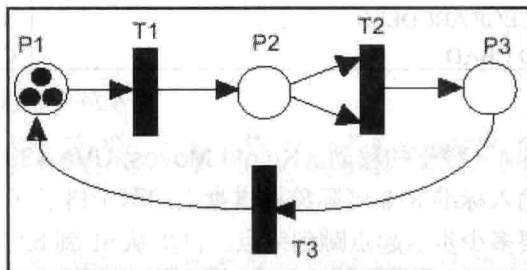


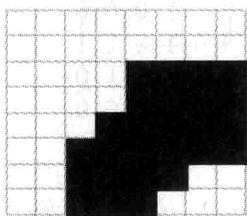
图 6-24 Petri 网模拟

输入一个 Petri 网络。初始时每个库所都有一个 token。每个变迁用一个整数序列表示，负数表示输入库所，正数表示输出库所。每个变迁至少包含一个输入和一个输出。最后输入一个整数 NF，表示要发生 NF 次变迁（同时有多个变迁允许时可以任选一个发生，输入保证这个选择不会影响最终结果）。

本题有一定实际意义，理解题意后编码并不复杂，建议读者一试。

习题 6-8 空间结构 (Spatial Structures, ACM/ICPC World Finals 1998, UVa806)

黑白图像有两种表示法：点阵表示和路径表示。路径表示法首先要把图像转化为四分树，然后记录所有黑结点到根的路径。例如，对于如图 6-25 所示的图像。



```

0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 1 1 1 1
0 0 0 0 1 1 1 1
0 0 0 1 1 1 1 1
0 0 1 1 1 1 1 1
0 0 1 1 1 1 0 0
0 0 1 1 1 0 0 0

```

1		2	3
		4	5
6	7	8	13
	9	10	
11	12	15	16
		17	18
			19

图 6-25 黑白图像

四分树如图 6-26 所示。

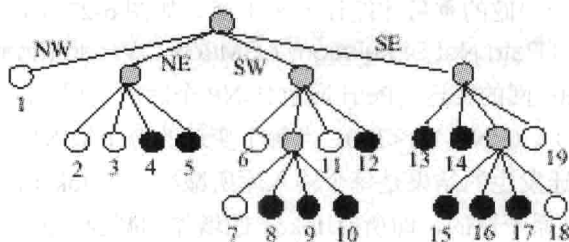


图 6-26 黑白图像四分树

NW、NE、SW、SE 分别用 1、2、3、4 表示。最后把得到的数字串看成是五进制的，转化为十进制后排序。例如上面的树在转化、排序后的结果是：9 14 17 22 23 44 63 69 88 94 113。

你的任务是在这两种表示法之间进行转换。在点阵表示法中，1 表示黑色，0 表示白色。图像总是正方形的，且长度 n 为 2 的整数幂，并满足 $n \leq 64$ 。输入输出细节请参见原题。

本题有一定实际意义，而且需要注意细节，建议读者一试。

习题 6-9 纸牌游戏 ("Accordian" Patience, UVa 127)

把 52 张牌从左到右排好，每张牌自成一个牌堆 (pile)。当某张牌与它左边那张牌或者左边第 3 张牌 “match” (花色 suit 或者点数 rank 相同) 时，就把这张牌移到那张牌上面。移动之后还要查看是否可以其他移动。只有位于牌堆顶部的牌才能移动或者参与 match。当牌堆之间出现空隙时要立刻把右边的所有牌堆左移一格来填补空隙。如果有多张牌可以移动，先移动最左边的那张牌；如果既可以移一格也可以移 3 格时，移 3 格。按顺序输入 52 张牌，输出最后的牌堆数以及各牌堆的牌数。

样例输入：

```
QD AD 8H 5S 3H 5H TC 4D JH KS 6H 8S JS AC AS 8D 2H QS TS 3S AH 4H TH TD
3C 6S
8C 7D 4C 4S 7S 9H 7C 5D 2S KD 2D QH JD 6D 9D JC 2C KH 3D QC 6C 9S KC 7H
9C 5C
AC 2C 3C 4C 5C 6C 7C 8C 9C TC JC QC KC AD 2D 3D 4D 5D 6D 7D 8D TD 9D JD
QD KD
AH 2H 3H 4H 5H 6H 7H 8H 9H KH 6S QH TH AS 2S 3S 4S 5S JH 7S 8S 9S TS JS
QS KS
#
```

样例输出：

```
6 piles remaining: 40 8 1 1 1 1
1 pile remaining: 52
```

习题 6-10 10-20-30 游戏 (10-20-30, ACM/ICPC World Finals 1996, UVa246)

有一种纸牌游戏叫做 10-20-30。游戏使用除大王和小王之外的 52 张牌，J、Q、K 的面值是 10，A 的面值是 1，其他牌的面值等于它的点数。

把 52 张牌叠放在一起放在手里，然后从最上面开始依次拿出 7 张牌从左到右摆成一条直线放在桌子上，每一张牌代表一个牌堆。每次取出手中最上面的一张牌，从左至右依次放在各个牌堆的最下面。当往最右边的牌堆放了一张牌以后，重新往最左边的牌堆上放牌。

如果当某张牌放在某个牌堆上后，牌堆的最上面两张和最下面一张牌的和等于 10、20 或者 30，这 3 张牌将会从牌堆中拿走，然后按顺序放回手中并压在最下面。如果没有出现这种情况，将会检查最上面一张和最下面两张牌的和是否为 10、20 或者 30，解决方法类似。如果仍然没有出现这种情况，最后检查最下面的 3 张牌的和，并用类似的方法处理。例如，如果某一牌堆中的牌从上到下依次是 5、9、7、3，那么放上 6 以后的布局如图 6-27 所示。

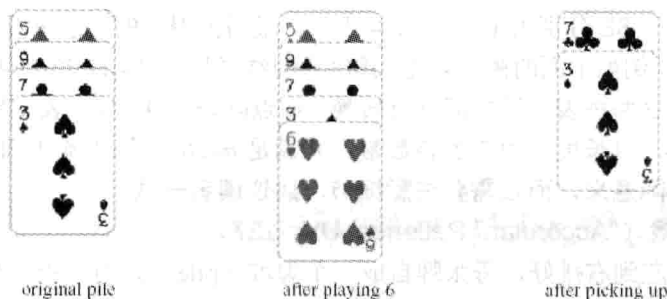


图 6-27 放上 6 后布局

如果放的不是 6，而是 Q，对应的情况如图 6-28 所示。

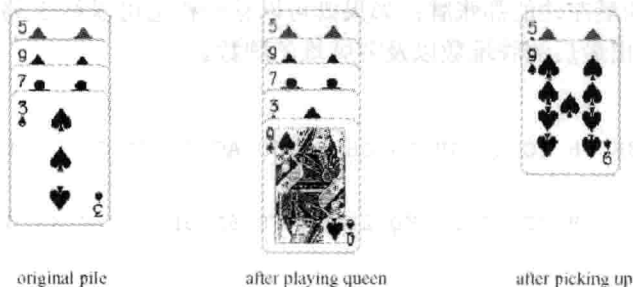


图 6-28 放上 Q 后布局

如果某次操作后某牌堆中没有剩下一张牌，那么将该牌堆便永远地清除掉，并把它右边的所有牌堆顺次往左移。如果所有牌堆都清除了，游戏胜利结束；如果手里没有牌了，游戏以失败告终；有时游戏永远无法结束，这时则称游戏出现循环。给出 52 张牌最开始在手中的顺序，请模拟这个游戏并计算出游戏结果。

习题 6-11 树重建 (Tree Reconstruction, UVa 10410)

输入一个 n ($n \leq 1000$) 结点树的 BFS 序列和 DFS 序列，你的任务是输出每个结点的子结点列表。输入序列（不管是 BFS 还是 DFS）是这样生成的：当一个结点被扩展时，其所有子结点应该按照编号从小到大的顺序访问。

例如，若 BFS 序列为 4 3 5 1 2 8 7 6，DFS 序列为 4 3 1 7 2 6 5 8，则一棵满足条件的树如图 6-29 所示。

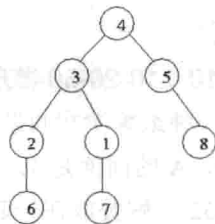


图 6-29 树重建

习题 6-12 筛子难题 (A Dicey Problem, ACM/ICPC World Finals 1999, UVa810)

图 6-30 (a) 是一个迷宫，图 6-30 (b) 是一个筛子。你的任务是把筛子放在起点（筛子顶面和正面的数字由输入给定），经过若干次滚动以后回到起点。

每次到达一个新格子时，格子上的数字必须和与它接触的筛子上的数字相同，除非到达的格子上画着五星（此时，与它接触的筛子上的数字可以任意）。输入一个 R 和 C 行 ($1 \leq R, C \leq 10$) 的迷宫、起点坐标以及顶面、正面的数字，输出一条可行的路径。

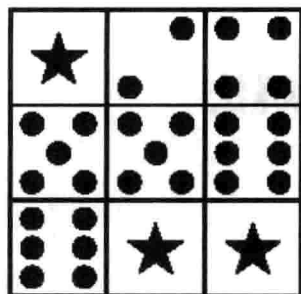


Figure 1: Sample Dice Maze

(a)

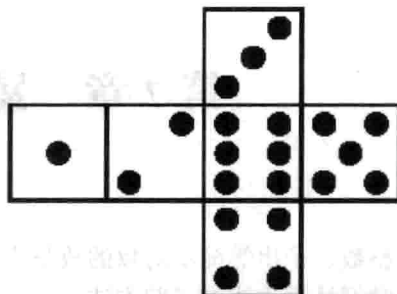


Figure 2: Standard Layout of Six-Sided Die

(b)

图 6-30 筛子难题

习题 6-13 电子表格计算器 (Spreadsheet Calculator, ACM/ICPC World Finals 1992, UVa215)

在一个 R 行 C 列 ($R \leq 20, C \leq 10$) 的电子表格中, 行编号为 A~T, 列编号为 0~9。按照行优先顺序输入电子表格的各个单元格。每个单元格可能是整数 (可能是负数) 或者引用了其他单元格的表达式 (只包含非负整数、单元格名称和加减号, 没有括号)。表达式保证以单元格名称开头, 内部不含空白字符, 且最多包含 75 个字符。

尽量计算出所有表达式的值, 然后输出各个单元格的值 (计算结果保证为绝对值不超过 10000 的整数)。如果某些单元格循环引用, 在表格之后输出 (仍按行优先顺序), 如图 6-31 所示。

样例输入	样例输出
2 2	0 1
A1+B1	A 3 5
5	B 3 -2
3	A0: A0
B0-A1	B0: C1
3 2	C1: B0+A1
A0	
5	
C1	
7	
A1+B1	
B0+A1	
0 0	

图 6-31 电子表格计算器输入与输出

习题 6-14 检查员的难题 (Inspector's Dilemma, ACM/ICPC Dhaka 2007, UVa12118)

某国家有 V ($V \leq 1000$) 个城市, 每两个城市之间都有一条双向道路直接相连, 长度为 T 。你的任务是找一条最短的道路 (起点和终点任意), 使得该道路经过 E 条指定的边。

例如, 若 $V=5, E=3, T=1$, 指定的 3 条边为 1-2、1-3 和 4-5, 则最优道路为 3-1-2-4-5, 长度为 $4 \times 1=4$ 。